Practical Assignment

House Price Prediction Using MLOps

Name	Dande Teja
Employee Code	8276
District	Namakkal
GitHub	github.com/tejadande123-cyber/ house-price-prediction-mlops

Objective: Develop a house-price prediction model and demonstrate an end-to-end MLOps workflow covering dataset preparation, preprocessing, Linear Regression training, evaluation, MLflow tracking, model saving, Git/GitHub version control, FastAPI deployment and Swagger testing.

1. Problem Statement

A real estate company wants to predict the price of a house from its area, number of bedrooms, number of bathrooms and age. This project implements the required Machine Learning model and connects the major stages through a basic MLOps workflow.

2. Dataset Preparation

Column	Description
area_sqft	House area in square feet
bedrooms	Number of bedrooms
bathrooms	Number of bathrooms
house_age	Age of house in years
price	House price in INR (target)

The five illustrative records supplied in the assignment are retained. The program generates **995 synthetic records**, resulting in **1,000 total records**.

```
OUTPUT = Path("data/house_prices.csv")
rng = np.random.default_rng(10)
n = 995

example_data = pd.DataFrame({
    "area_sqft": [1000, 1500, 2000, 2500, 1200],
    "bedrooms": [2, 3, 3, 4, 2],
    "bathrooms": [1, 2, 2, 3, 2],
    "house_age": [5, 3, 8, 2, 10],
    "price": [
        3500000, 5500000, 6500000,
        9500000, 3800000
    ]
})
```

Synthetic Price Logic

```
price = (
    500000
    + 2800 * area_sqft
    + 250000 * bedrooms
    + 300000 * bathrooms
    - 40000 * house_age
    + noise
)
```

The price relationship is an assumed formula for this synthetic practical dataset and is not a real-world valuation formula.

3. Data Preprocessing

The independent variables X are area_sqft, bedrooms, bathrooms and house_age. The dependent variable y is price. The data is split into 80% training and 20% testing.

```
FEATURES = [
    "area_sqft",
    "bedrooms",
    "bathrooms",
    "house_age"
]
TARGET = "price"

X = df[FEATURES]
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=10
)
```

Total: 1,000 records **Training:** 800 **Testing:** 200

4. Model Development

```
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

The Linear Regression model learns coefficients from the 800 training records and then generates price predictions for the 200 unseen test records.

5. Model Evaluation

```
mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, predictions)
```

Metric	Result
MAE	INR 316,895.84
RMSE	INR 402,549.35
R2	0.9805

MAE is the average absolute error. RMSE gives more weight to large errors. R2 shows how much of the target variation is explained by the model.

6. MLflow Experiment Tracking

The MLflow experiment is named **House Price Prediction**. The run records model parameters, dataset split information, evaluation metrics and the trained model artifact.

```
mlflow.set_experiment(EXPERIMENT_NAME)

with mlflow.start_run(
    run_name="Linear_Regression_Run"
):
    mlflow.log_param(
        "model_type",
        "Linear Regression"
    )
    mlflow.log_param("test_size", TEST_SIZE)
    mlflow.log_param(
        "random_state",
        RANDOM_STATE
    )
    mlflow.log_param(
        "total_records",
        len(df)
    )
    mlflow.log_param(
        "training_records",
        len(X_train)
    )
    mlflow.log_param(
        "testing_records",
        len(X_test)
    )

    mlflow.log_metric("mae", mae)
    mlflow.log_metric("rmse", rmse)
    mlflow.log_metric("r2", r2)
```

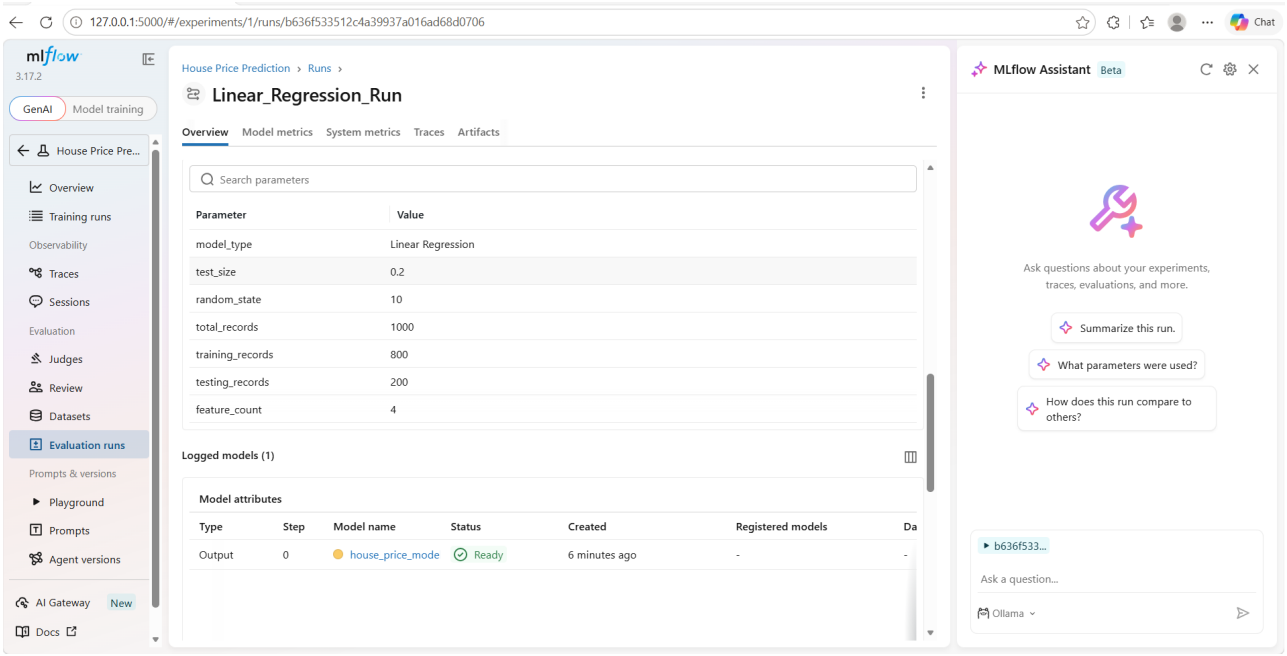


Figure 1. MLflow run overview showing parameters and the logged model artifact.

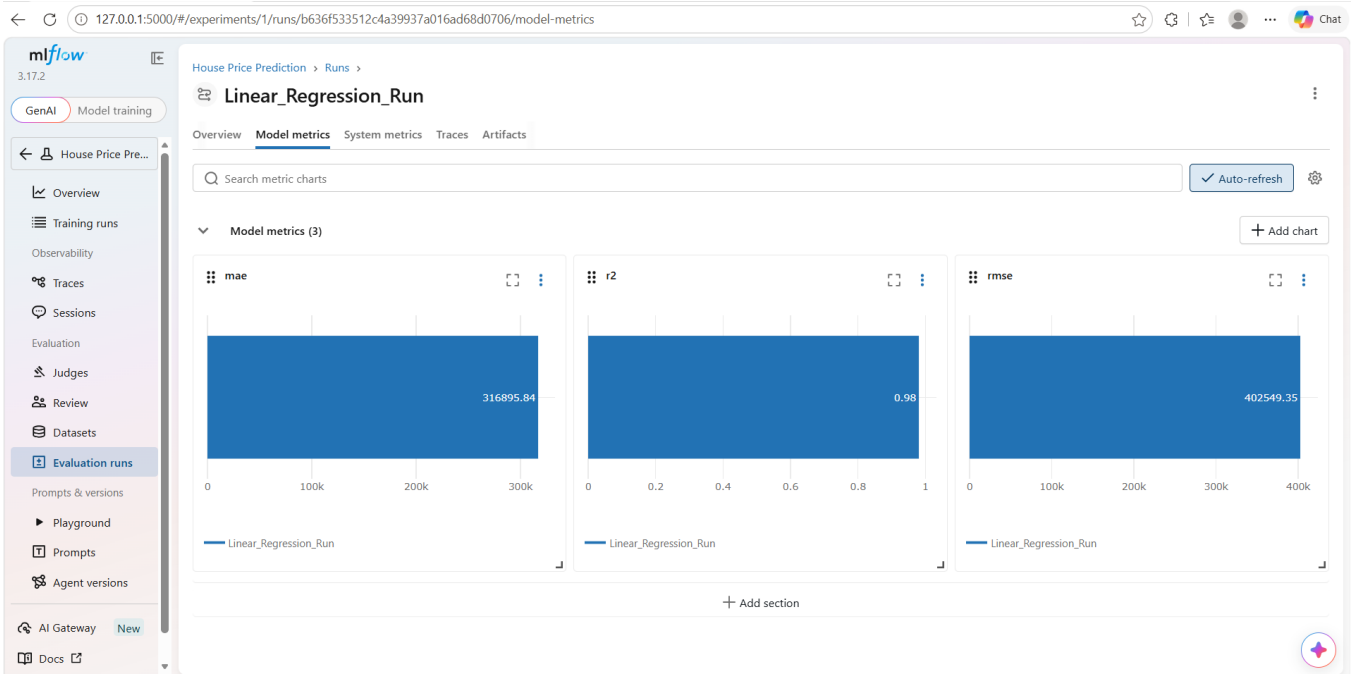


Figure 2. MLflow metrics showing MAE, R2 and RMSE.

7. Model Versioning and Persistence

The trained model is saved with Joblib as **models/house_model.joblib**. Git is used for source-code version control, and the project is hosted on GitHub.

```
MODEL_PATH.parent.mkdir(
    parents=True,
    exist_ok=True
)
joblib.dump(model, MODEL_PATH)
```

GitHub: https://github.com/tejadande123-cyber/house-price-prediction-mlops

8. FastAPI Deployment

The FastAPI application loads the saved model, validates the input using Pydantic and exposes a POST /predict endpoint.

```
model = joblib.load(MODEL_PATH)

class HouseInput(BaseModel):
    area_sqft: float = Field(gt=0)
    bedrooms: int = Field(ge=1)
    bathrooms: int = Field(ge=1)
    house_age: float = Field(ge=0)

@app.post("/predict")
def predict(house: HouseInput):
    row = pd.DataFrame(
        [house.model_dump()],
        columns=FEATURES
    )
    prediction = float(
        model.predict(row)[0]
    )
    return {
        "predicted_price": round(
            prediction, 2
        ),
        "currency": "INR"
    }
```

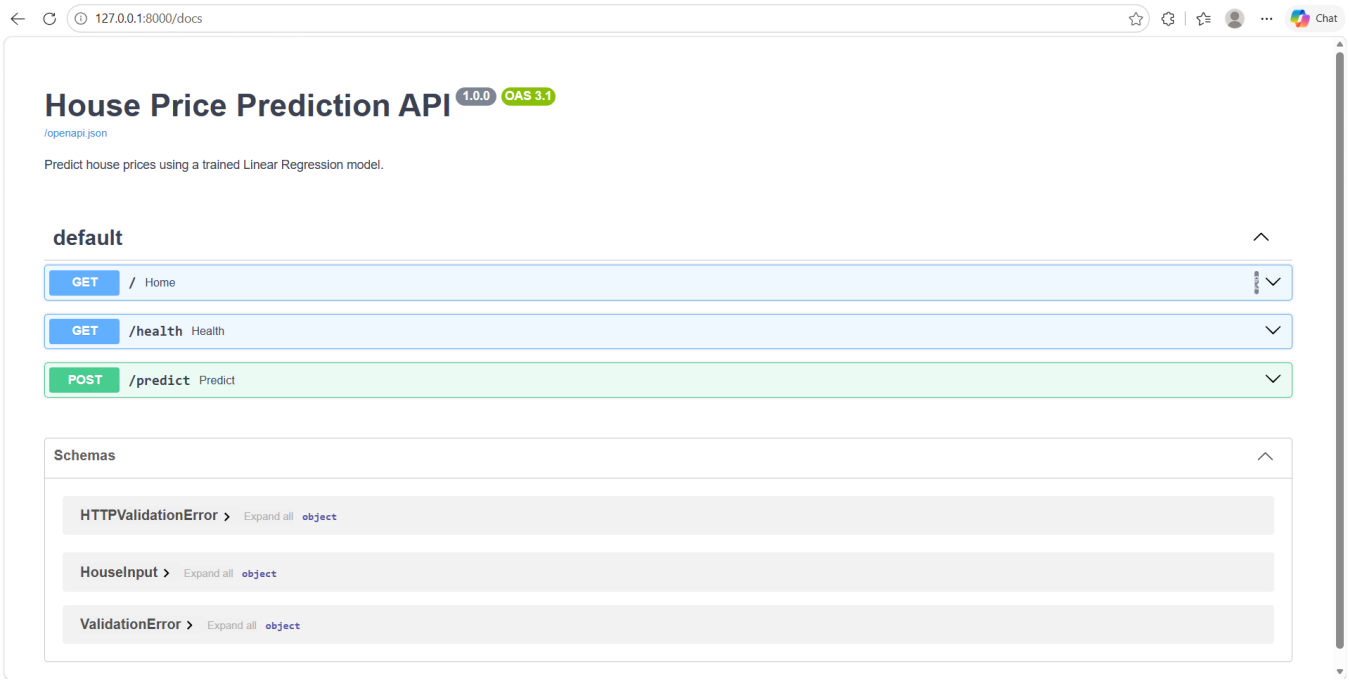


Figure 3. FastAPI Swagger UI showing Home, Health and Predict endpoints.

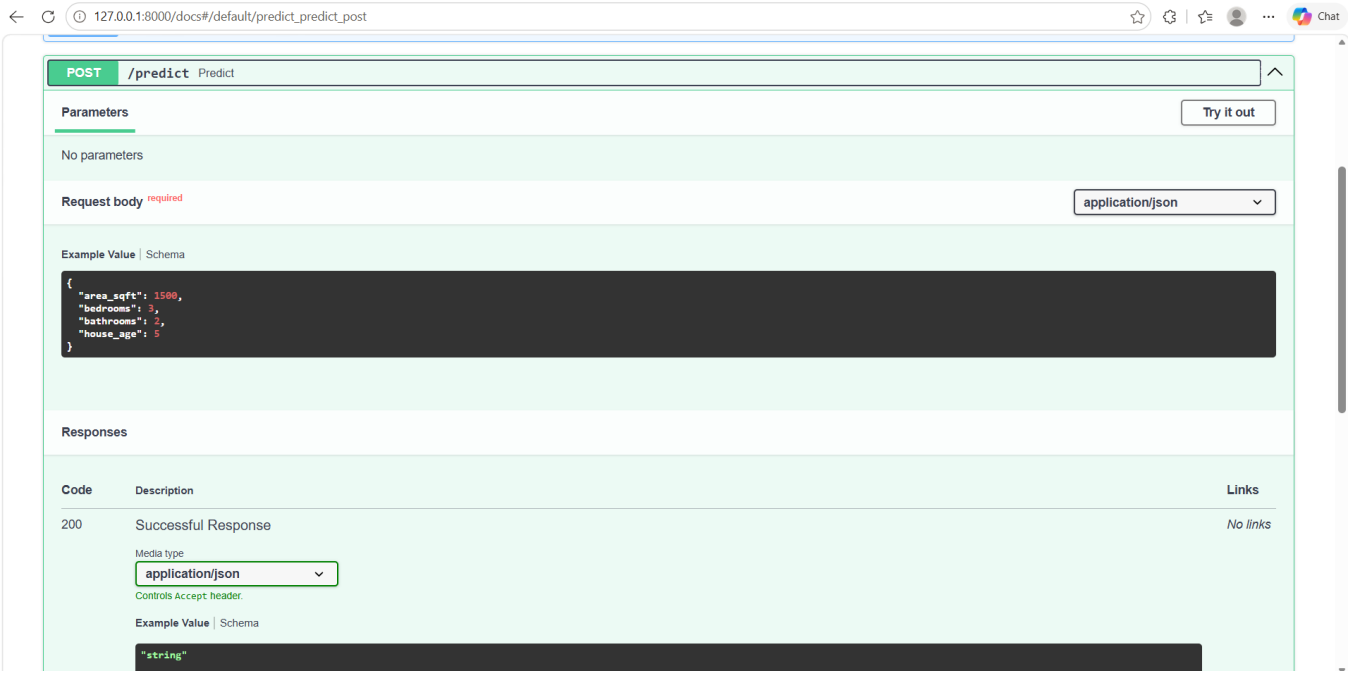


Figure 4. POST /predict request body in Swagger UI.

Example input: 1500 sq.ft area, 3 bedrooms, 2 bathrooms and house age 5 years.

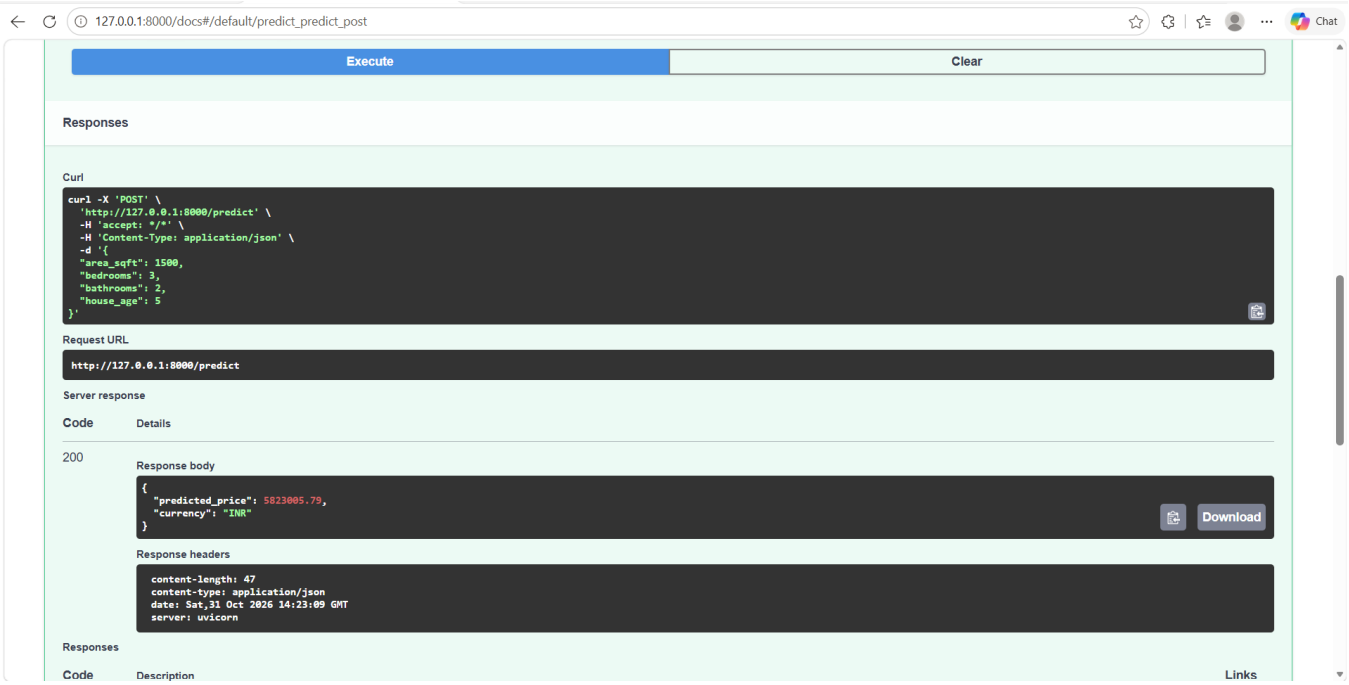


Figure 5. Successful HTTP 200 response with predicted price INR 5,823,005.79.

9. Successful End-to-End Execution

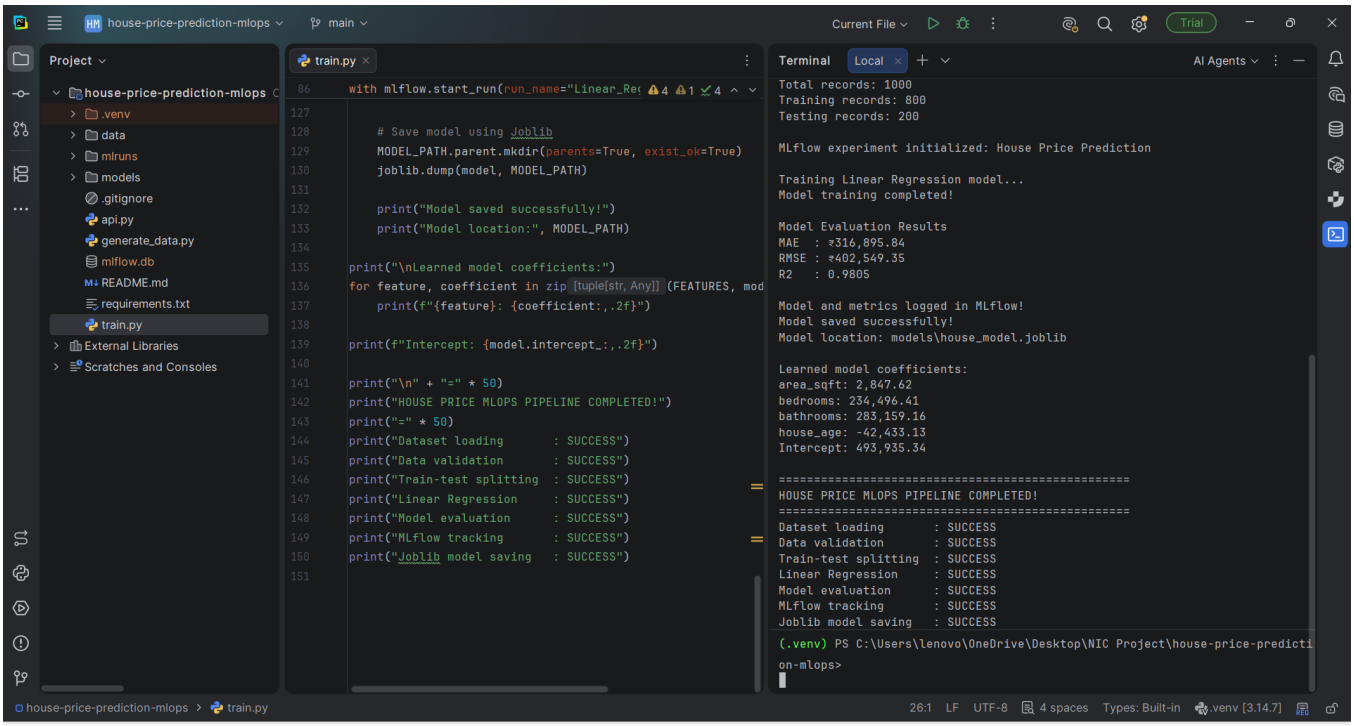


Figure 6. PyCharm project and successful training pipeline output.

The terminal confirms successful dataset loading, validation, train-test splitting, Linear Regression training, evaluation, MLflow tracking and Joblib model saving.

10. Project Structure

```
house-price-prediction-mlops/
|
+-- data/
|   +-- house_prices.csv
|
+-- models/
|   +-- house_model.joblib
|
+-- generate_data.py
+-- train.py
+-- api.py
+-- requirements.txt
+-- .gitignore
+-- README.md
```

11. Practical Task Completion Summary

Task	Completed Work
Task 1	Created house_prices.csv, used Pandas, displayed the first five records, and checked missing values.
Task 2	Selected X and y and split the dataset into 80% training data and 20% testing data.
Task 3	Trained Linear Regression, generated predictions, and calculated MAE, RMSE and R2.
Task 4	Created the MLflow experiment, logged parameters and metrics, and saved the trained model artifact.
Task 5	Saved the model with Joblib, maintained source code using Git/GitHub, and recorded dependencies in requirements.txt.
Task 6	Created the FastAPI application, accepted house details, returned the predicted price, and tested the API using Swagger UI.

12. How to Run

```
python -m pip install -r requirements.txt
python generate_data.py
python train.py

mlflow ui
uvicorn api:app --reload
```

MLflow UI: http://127.0.0.1:5000
Swagger UI: http://127.0.0.1:8000/docs

13. Conclusion

The assignment demonstrates the complete basic MLOps workflow from dataset generation to a deployed prediction API. The model achieved an R2 of approximately **0.98** on the synthetic exercise dataset. Since the dataset is synthetic, the result demonstrates the workflow and should not be interpreted as a real-world property valuation benchmark.